Department of Information Engineering Technology
The Superior University, Lahore

Mobile Application Development Lab

Experiment No.9
Flutter Tab Bar and Drawer

Prepared for

Eisha Nawaz

By:

Name: _____

ID: _____

Section: _____

Semester: _____

Total Marks: 10

Obtained Marks: _____

Signature: _____

Date: _____

Experiment No.9

Flutter Tab Bar and Drawer

(Rubrics)

Name: _____ **Roll No:** _____

A. PSYCHOMOTOR

Sr. No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Follow Procedures	1	0	0.25	0.5	0.75	1	
2	Software and Simulations	2	0	0.5	1	1.5	2	
3	Accuracy in Output Results	3	0	0.75	1.5	2.25	3	
Sub Total		**6**	**Sub Total marks Obtained in Psychomotor(P)**					

B. AFFECTIVE

Sr. No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Respond to Questions	1	0	0.25	0.5	0.75	1	
2	Lab Report	1	0	0.25	0.5	0.75	1	
3	Assigned Task	2	0	0.5	1	1.5	2	
Sub Total		**4**	**Sub Total marks Obtained in Affective (A)**					

Instructor Name: Eisha Nawaz **Total Marks (P+A):** 10

Instructor Signature: _____ **Obtained Marks (P+A):** _____

USING THE TABBAR AND TABBARVIEW

The `TabBar` widget is a Material Design widget that displays a horizontal row of tabs. The `tabs` property takes a `List` of `Widgets`, and you add tabs by using the `Tab` widget. Instead of using the `Tab` widget, you could create a custom widget, which shows the power of Flutter. The selected `Tab` is marked with a bottom selection line.

The `TabBarView` widget is used in conjunction with the `TabBar` widget to display the page of the selected tab. Users can swipe left or right to change content or tap each `Tab`.

Both the `TabBar` (Figure 8.3) and `TabBarView` widgets take a `controller` property of `TabController`. The `TabController` is responsible for syncing tab selections between a `TabBar` and a `TabBarView`. Since the `TabController` syncs the tab selections, you need to declare the `SingleTickerProviderStateMixin` to the class. In Chapter 7, "Adding Animation to an App," you learned how to implement the `Ticker` class that is driven by the `ScheduleBinding.scheduleFrameCallback` reporting once per animation frame. It is trying to sync the animation to be as smooth as possible.

FIGURE 8.3: TabBar in the Scaffold bottomNavigationBar property

TRY IT OUT Creating the TabBar and TabBarView App

In this example, the `TabBar` widget is the child of a `bottomNavigationBar` property. This places the `TabBar` at the bottom of the screen, but you could also place it in the `AppBar` or a custom location. When you use a `TabBar` in combination with the `TabBarView`, once a `Tab` is selected, it automatically displays the appropriate content. In this project, the content is represented by three separate pages. You'll create the same three pages as you did in the `BottomNavigationBar` project.

1. Create a new Flutter project and name it `ch8_tabbar`. Once more, you can follow the instructions in Chapter 4. For this project, you need to create only the `pages` folder.
2. Open the `home.dart` file, and add to the `body` a `SafeArea` with `TabBarView` as a `child`. The `TabBarView controller` property is a `TabController` variable called `_tabController`. Add to the `TabBarView children` property the `Birthdays()`, `Gratitude()`, and `Reminders()` pages that you'll create in step 3.

```
body: SafeArea(
  child: TabBarView(
```

```
      controller:  _tabController,
      children: [
        Birthdays(),
        Gratitude(),
        Reminders(),
      ],
    ),
  ),
```

3. This time you'll create the pages that you are navigating to first. Like in the `BottomNavigationBar` app, create three `StatelessWidget` pages and call them `Birthdays`, `Gratitude`, and `Reminders`. Each page has a `Scaffold` with `Center()` for the `body`. The `Center child` is an `Icon` with the `size` of 120.0 pixels and a `color`.

The following are the three Dart files, `birthdays.dart`, `gratitude.dart`, and `reminders.dart`:

```
// birthdays.dart
import 'package:flutter/material.dart';

class Birthdays extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Icon(
          Icons.cake,
          size: 120.0,
          color: Colors.orange,
        ),
      ),
    );
  }
}

// gratitude.dart
import 'package:flutter/material.dart';

class Gratitude extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Icon(
          Icons.sentiment_satisfied,
          size: 120.0,
          color: Colors.lightGreen,
        ),
      ),
    );
  }
}

// reminders.dart
import 'package:flutter/material.dart';
```

```
class Reminders extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Icon(
          Icons.access_alarm,
          size: 120.0,
          color: Colors.purple,
        ),
      ),
    );
  }
}
```

4. Import each page in the `home.dart` file.

```
import 'package:flutter/material.dart';
import 'birthdays.dart';
import 'gratitude.dart';
import 'reminders.dart';
```

5. Declare the `TickerProviderStateMixin` to the `_HomeState` class by adding `with TickerProviderStateMixin`. The `AnimationController` `vsync` argument will use it.

```
class _HomeState extends State<Home> with SingleTickerProviderStateMixin {...}
```

6. Declare a `TabController` variable by the name of `_tabController`. Override the `initState()` method to initialize the `_tabController` with the `vsync` argument and a `length` value of 3. The `vsync` is referenced by `this`, meaning this reference of the `_HomeState` class. The `length` represents the number of `Tab`s to show. Add a `Listener` to the `_tabController` to catch when a `Tab` is changed. Then override the `dispose()` method for when the page closes to properly dispose of the `_tabController`.

   Note in the `_tabChanged` method you check for `indexIsChanging` before showing which `Tab` is tapped. If you do not check for `indexIsChanging`, then the code runs twice.

```
class _HomeState extends State<Home> with SingleTickerProviderStateMixin {
  TabController _tabController;

  @override
  void initState() {
    super.initState();

    _tabController = TabController(vsync: this, length: 3);
    _tabController.addListener(_tabChanged);
  }

  @override
  void dispose() {
    super.dispose();

  @override
  void dispose() {
    _tabController.dispose();
```

```
    super.dispose();
  }
    _tabController.dispose();
  }

  void _tabChanged() {
    // Check if Tab Controller index is changing, otherwise we get the notice twice
    if (_tabController.indexIsChanging) {
      print('tabChanged: ${_tabController.index}');
    }
  }
```

7. Add a `TabBar` as a `child` of the `bottomNavigationBar` `Scaffold` property.

```
bottomNavigationBar: SafeArea(
  child: TabBar(),
),
```

8. Pass the `_tabController` for the `TabBar` `controller` property. I customized the `labelColor` and `unselectedLabelColor` by using, respectively, `Colors.black54` and `Colors.black38`, but feel free to experiment using different colors.

```
bottomNavigationBar: SafeArea(
  child: TabBar(
    controller: _tabController,
    labelColor: Colors.black54,
    unselectedLabelColor: Colors.black38,
  ),
),
```

9. Add three `Tab` widgets to the `tabs` widget list. Customize each `Tab` `icon` and `text`.

```
bottomNavigationBar: SafeArea(
  child: TabBar(
    controller: _tabController,
    labelColor: Colors.black54,
    unselectedLabelColor: Colors.black38,
    tabs: [
      Tab(
        icon: Icon(Icons.cake),
        text: 'Birthdays',
      ),
      Tab(
        icon: Icon(Icons.sentiment_satisfied),
        text: 'Gratitude',
      ),
      Tab(
        icon: Icon(Icons.access_alarm),
        text: 'Reminders',
      ),
    ],
  ),
),
```

How It Works

When `TabBar` and `TabBarView` are used together, the correct associated page is automatically loaded. When the user swipes the `TabBarView` left or right, it scrolls to the correct page and selects the corresponding tab in the `TabBar`. All of these powerful features are built in; no custom coding is necessary.

How does it know which page belongs to which tab? The `TabController` is responsible for syncing tab selections between a `TabBar` and a `TabBarView`. Since the `TabController` syncs the tab selections, you need to declare the `SingleTickerProviderStateMixin` to the class.

Both the `TabBar` and `TabBarView` use the same `TabController`. The `TabController` is initiated by passing a `vsync` argument and a `length` argument. The `length` argument is the number of tabs to show. An optional `TabController Listener` is added to listen to `Tab` changes and take appropriate action if necessary, perhaps saving data before a `Tab` is switched. Each `Tab` is added to the `TabBar tabs` widget list by customizing each `icon` and `text`.

`TabBarView` is responsible for loading the appropriate page when `Tab` selection changes. The page views are listed as the `children` property of the `TabBarView` widget.

USING THE DRAWER AND LISTVIEW

You might be wondering why I'm covering the `ListView` in this navigation chapter. Well, it works great with the `Drawer` widget. `ListView` widgets are used quite often for selecting an item from a list to navigate to a detailed page.

`Drawer` is a Material Design panel that slides horizontally from the left or right edge of the `Scaffold`, the device screen. `Drawer` is used with the `Scaffold drawer` (left-side) property or `endDrawer` (right-side) property. `Drawer` can be customized for each individual need but usually has a header to show an image or fixed information and a `ListView` to show a list of navigable pages. Usually, a `Drawer` is used when the navigation list has many items.

To set the `Drawer` header, you have two built-in options, the `UserAccountsDrawerHeader` or the `DrawerHeader`. The `UserAccountsDrawerHeader` is intended to display the app's user details by setting the `currentAccountPicture`, `accountName`, `accountEmail`, `otherAccountsPictures`, and `decoration` properties.

```
// User details
UserAccountsDrawerHeader(
  currentAccountPicture: Icon(Icons.face,),
  accountName: Text('Sandy Smith'),
  accountEmail: Text('sandy.smith@domainname.com'),
  otherAccountsPictures: <Widget>[
    Icon(Icons.bookmark_border),
  ],
  decoration: BoxDecoration(
    image: DecorationImage(
      image: AssetImage('assets/images/home_top_mountain.jpg'),
      fit: BoxFit.cover,
    ),
  ),
),
```

`DrawerHeader` is intended to display generic or custom information by setting the `padding`, `child`, `decoration`, and other properties.

```
// Generic or custom information
DrawerHeader(
  padding: EdgeInsets.zero,
  child: Icon(Icons.face),
  decoration: BoxDecoration(color: Colors.blue),
),
```

The standard `ListView` constructor allows you to build a short list of items quickly. The next chapter will go into more depth on how to use the ListView. see Figure

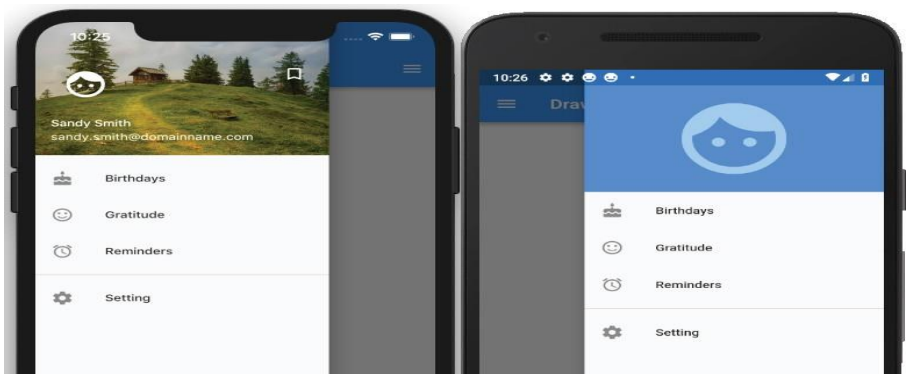


FIGURE 8.4: Drawer and ListView

TRY IT OUT Creating the Drawer App

In this example, the `Drawer` is added to the `drawer` or `endDrawer` property of the `Scaffold`. The `drawer` and `endDrawer` properties slide the `Drawer` from either left to right (`TextDirection.ltr`) or right to left (`TextDirection.rtl`). In this example, you'll add both the `drawer` and the `endDrawer` to show how to use both. You use the `UserAccountsDrawerHeader` to the `drawer` (left-side) property and the `DrawerHeader` to the `endDrawer` (right-side) property.

You use the `ListView` to add the `Drawer` content and `ListTile` to align text and icons for the menu list easily. In this project, you use the standard `ListView` constructor since you have a small list of menu items.

1. Create a new Flutter project and name it `ch8_drawer`, following the instructions in Chapter 4. For this project, you need to create the `pages`, `widgets`, and `assets/images` folders. Copy the `home_top_mountain.jpg` image to the `assets/images` folder.

2. Open the `pubspec.yaml` file and under `assets` add the `images` folder.

```
# To add assets to your application, add an assets section, like this:
assets:
  - assets/images/
```

3. Add the folder `assets` and subfolder `images` at the project's root and then copy the `home_top_mountain.jpg` file to the `images` folder.

4. Click the Save button; depending on the editor you are using, it automatically runs the `flutter packages get`. Once finished, it shows a message of `Process finished with exit code 0`.

   If it does not automatically run the command for you, open the Terminal window (located at the bottom of your editor) and type `flutter packages get`.

5. Create the pages that you are navigating to first. Create three `StatelessWidget` pages and call them `Birthdays`, `Gratitude`, and `Reminders`. Each page has a `Scaffold` with `Center()` for the `body`. The `Center child` is an `Icon` with the `size` value of 120.0 pixels and a `color` property.

```
class Birthdays extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Icon(
          Icons.cake,
          size: 120.0,
          color: Colors.orange,
        ),
      ),
    );
  }
}
```

6. Add the `AppBar` to the `Scaffold`, which is needed for navigating back to the home page. The following shows the three Dart files, `birthdays.dart`, `gratitude.dart`, and `reminders.dart`:

```
// birthdays.dart
import 'package:flutter/material.dart';
```

```
class Birthdays extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Birthdays'),
      ),
      body: Center(
        child: Icon(
          Icons.cake,
          size: 120.0,
          color: Colors.orange,
        ),
      ),
    );
  }
}
```

```
// gratitude.dart
import 'package:flutter/material.dart';

class Gratitude extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Gratitude'),
      ),
      body: Center(
        child: Icon(
          Icons.sentiment_satisfied,
          size: 120.0,
          color: Colors.lightGreen,
        ),
      ),
    );
  }
}
```

```
// reminders.dart
import 'package:flutter/material.dart';

class Reminders extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Reminders'),
      ),
      body: Center(
        child: Icon(
          Icons.access_alarm,
          size: 120.0,
          color: Colors.purple,
        ),
      ),
    );
  }
}
```

7. The left and right `Drawer` widgets share the same menu list, and you'll write it first. Create a new

Dart file in the `widgets` folder. Right-click the `widgets` folder and then select New ➪ Dart File, enter `menu_list_tile.dart`, and click the OK button to save.

8. Import the `material.dart`, `birthdays.dart`, `gratitude.dart`, and `reminders.dart` classes (pages). Add a new line and then start typing **st**; the autocompletion help opens, so select the `stful` abbreviation and give it a name of `MenuListTileWidget`.

```
import 'package:flutter/material.dart';
import  'package:ch8_drawer/pages/birthdays.dart';
import  'package:ch8_drawer/pages/gratitude.dart';
import 'package:ch8_drawer/pages/reminders.dart';
```

9. The `Widget build(BuildContext context)` returns a `Column`. The `Column children` widget list contains multiple `ListTile`s representing each menu item. Add a `Divider` widget with the `color` property set to `Colors.grey` before the last `ListTile` widget.

```
@override
Widget build(BuildContext context) {
  return Column(
    children: <Widget>[
      ListTile(),
      ListTile(),
      ListTile(),
      Divider(color: Colors.grey),
      ListTile(),
    ],
  );
}
```

10. For each `ListTile`, set the `leading` property as an `Icon` and the `title` property as a `Text`. For the `onTap` property, you first call `Navigator.pop()` to close the open `Drawer` and then call `Navigator.push()` to open the selected page.

```
@override
Widget build(BuildContext context) {
  return Column(
    children: <Widget>[
      ListTile(
        leading: Icon(Icons.cake),
        title: Text('Birthdays'),
        onTap: () {
          Navigator.pop(context);
          Navigator.push(
            context,
            MaterialPageRoute(
              builder: (context) => Birthdays(),
            ),
          );
        },
      ),
      ListTile(
        leading: Icon(Icons.sentiment_satisfied),
        title: Text('Gratitude'),
        onTap: () {
          Navigator.pop(context);
          Navigator.push(
            context,
            MaterialPageRoute(
              builder: (context) => Gratitude(),
            ),
```

```
        );
      },
    ),
    ListTile(
      leading: Icon(Icons.alarm),
      title: Text('Reminders'),

            onTap: () {
              Navigator.pop(context);
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => Reminders(),
                ),
              );
            },
          ),
          Divider(color: Colors.grey),
          ListTile(
            leading: Icon(Icons.settings),
            title: Text('Setting'),
            onTap: () {
              Navigator.pop(context);
            },
          ),
        ],
      );
    }
```

11. Create a new Dart file in the `widgets` folder. Right-click the `widgets` folder and then select New ➪ Dart File, enter `left_drawer.dart`, and click the OK button to save.

12. Import the `material.dart` library and the `menu_list_tile.dart` class.

```
import 'package:flutter/material.dart';
import 'package:ch8_drawer/widgets/menu_list_tile.dart';
```

13. Add a new line and then start typing **st**; the autocompletion help opens. Select the `stful` abbreviation and give it a name of `LeftDrawerWidget`.

```
class LeftDrawerWidget extends StatelessWidget {
  const LeftDrawerWidget({
    Key key,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Drawer();
  }
}
```

14. The `Widget build(BuildContext context)` returns a `Drawer`. The `Drawer child` is a `ListView children` list widget of a `UserAccountsDrawerHeader` widget and a call to the `const MenuListTileWidget()` widget class. To fill the entire `Drawer` space, you set the `ListView padding` property to `EdgeInsets.zero`.

```
@override
Widget build(BuildContext context) {
  return Drawer(
```

```
            child: ListView(
              padding: EdgeInsets.zero,

        children: <Widget>[
          UserAccountsDrawerHeader(),
          const MenuListTileWidget(),
        ],
      ),
    );
  }
```

15. For the `UserAccountsDrawerHeader`, set the `currentAccountPicture`, `accountName`, `accountEmail`, `otherAccountsPictures`, and `decoration` properties.

```
@override
Widget build(BuildContext context) {
  return Drawer(
    child: ListView(
      padding: EdgeInsets.zero,
      children: <Widget>[
        UserAccountsDrawerHeader(
          currentAccountPicture: Icon(
            Icons.face,
            size: 48.0,
            color: Colors.white,
          ),
          accountName: Text('Sandy Smith'),
          accountEmail: Text('sandy.smith@domainname.com'),
          otherAccountsPictures: <Widget>[
            Icon(
              Icons.bookmark_border,
              color: Colors.white,
            )
          ],
          decoration: BoxDecoration(
            image: DecorationImage(
              image: AssetImage('assets/images/home_top_mountain.jpg'),
              fit: BoxFit.cover,
            ),
          ),
        ),
        const MenuListTileWidget(),
      ],
    ),
  );
}
```

16. Create a new Dart file in the `widgets` folder. Right-click the `widgets` folder and then select New ⇨ Dart File, enter `right_drawer.dart`, and click the OK button to save. Import the `material.dart` library and the `menu_list_tile.dart` class.

```
import 'package:flutter/material.dart';
import 'package:ch8_drawer/widgets/menu_list_tile.dart';
```

17. Add a new line and then start typing **st**; the autocompletion help opens. Select the `stful` abbreviation and give it a name of `RightDrawerWidget`.

```
class RightDrawerWidget extends StatelessWidget {
  const RightDrawerWidget({
    Key key,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Drawer();
  }
}
```

18. The `Widget build(BuildContext context)` returns a `Drawer`. The `Drawer child` is a `ListView children` list of a `DrawerHeader` widget and a call to the `const MenuListTileWidget()` widget class. To fill the entire `Drawer` space, you set the `ListView padding` property to `EdgeInsets.zero`.

```
@override
Widget build(BuildContext context) {
  return Drawer(
    child: ListView(
      padding: EdgeInsets.zero,
      children: <Widget>[
        DrawerHeader(),
        const MenuListTileWidget(),
      ],
    ),
  );
}
```

19. For the `DrawerHeader`, set the `padding`, `child`, and `decoration` properties.

```
@override
Widget build(BuildContext context) {
  return Drawer(
    child: ListView(
      padding: EdgeInsets.zero,
      children: <Widget>[
        DrawerHeader(
          padding: EdgeInsets.zero,
          child: Icon(
            Icons.face,
            size: 128.0,
            color: Colors.white54,
          ),
          decoration: BoxDecoration(color: Colors.blue),
        ),
        const MenuListTileWidget(),
      ],
    ),
  );
}
```

20. Open the `home.dart` file and import the `material.dart`, `birthdays.dart`, `gratitude.dart`, and `reminders.dart` classes.

```
import 'package:flutter/material.dart';
import 'birthdays.dart';
import 'gratitude.dart';
import 'reminders.dart';
```

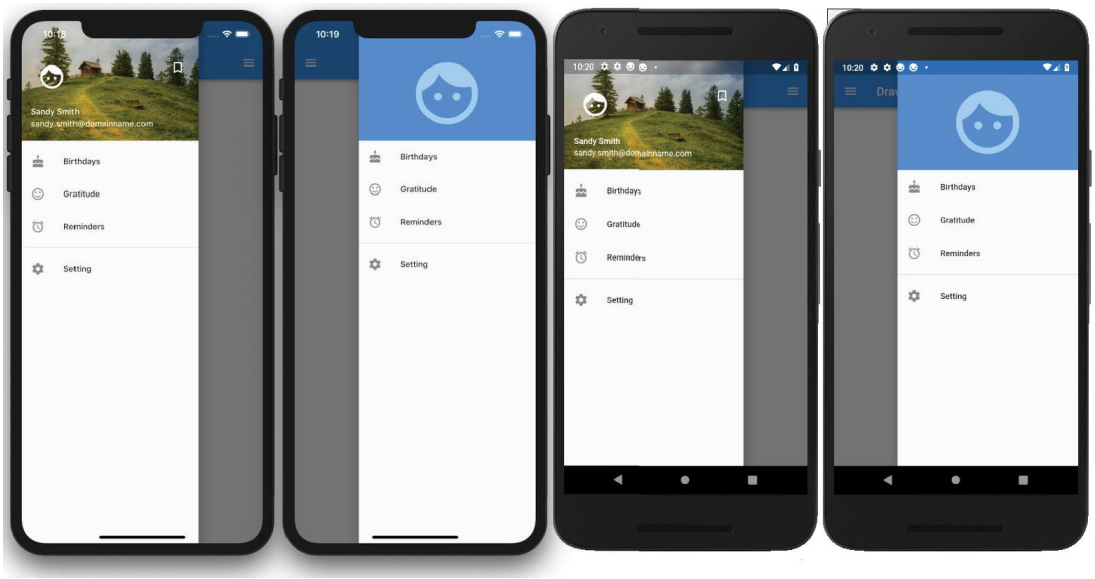
21. Add to the `body` a `SafeArea` with a `Container` as a `child`.

```
body: SafeArea(
  child: Container(),
),
```

22. Add to the `Scaffold drawer` property a call to the `LeftDrawerWidget()` widget class and for the `endDrawer` property a call to the `RightDrawerWidget()` widget class.

Note that you use the `const` keyword before calling each widget class to take advantage of caching and subtree rebuilding for better performance.

```
return Scaffold(
  appBar: AppBar(
    title: Text('Drawer'),
  ),
  drawer: const LeftDrawerWidget(),
  endDrawer: const RightDrawerWidget(),
  body: SafeArea(
    child: Container(),
  ),
);
```



How It Works

To add a `Drawer` to an app, you set the `Scaffold drawer` or `endDrawer` property. The `drawer` and `endDrawer` properties slide the `Drawer` from either left to right (`TextDirection.ltr`) or right to left (`TextDirection.rtl`).

The `Drawer` widget takes a `child` property, and you passed a `ListView`. Using a `ListView` allows you to create a scrollable menu list of items. For the `ListView children` widget list, you created two widget classes, one to build the `Drawer` header and one to build the list of menu items. To set the `Drawer` header, you have two options, `UserAccountsDrawerHeader` or `DrawerHeader`. These two widgets easily allow you to set header content depending on the requirements. You looked at two examples for the `Drawer` header by calling the appropriate widget class `LeftDrawerWidget()` or `RightDrawerWidget()`.

For the menu items list, you used the `MenuListTileWidget()` widget class. This class returns a `Column` widget that uses the `ListTile` to build your menu list. The `ListTile` widget allows you to set the `leading Icon`, `title`, and `onTap` properties. The `onTap` property calls `Navigator.pop()` to close `Drawer` and calls `Navigator.push()` to navigate to the selected page.

This app is an excellent example of creating a shallow widget tree by using widget classes and separat- ing them into individual files for maximum reuse. You also nested widget classes with the left and right widget classes both calling the menu list class.

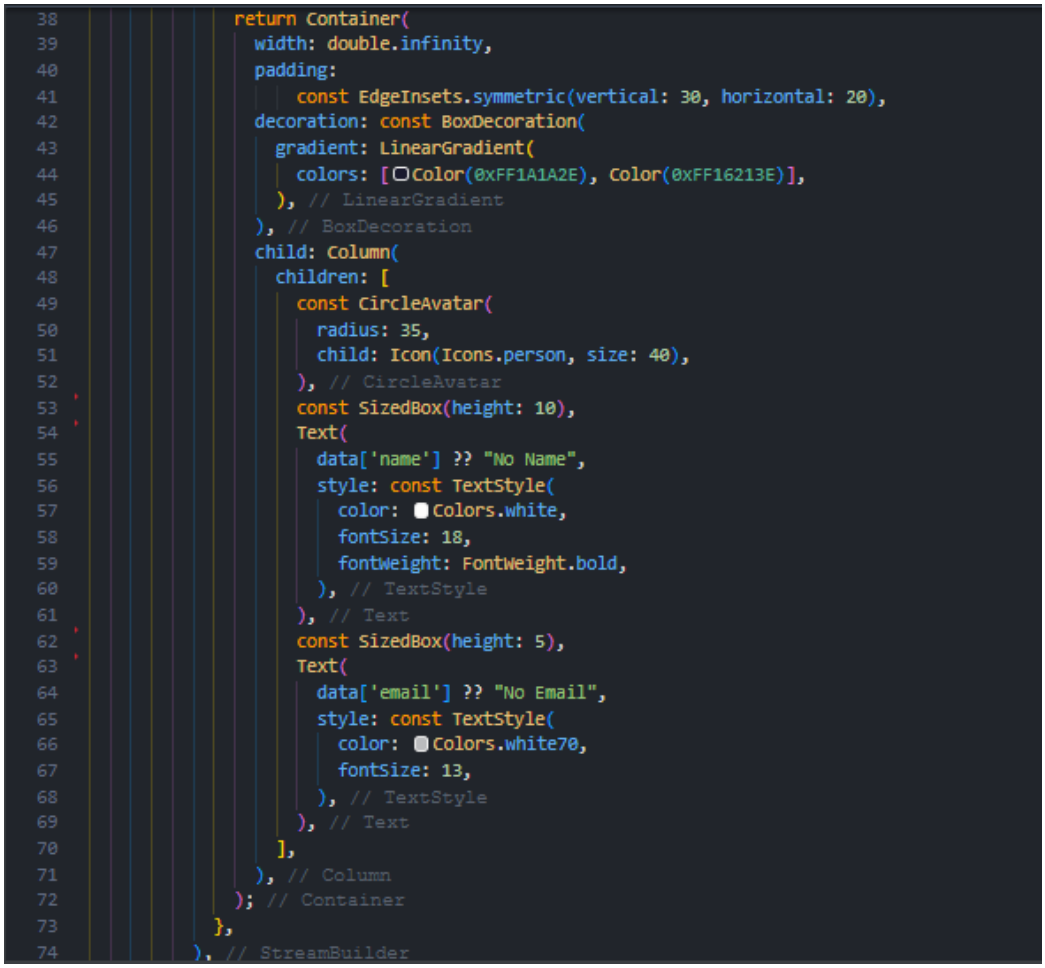
Solution:

Tab Bar:

```
body: TabBarView(
  controller: _tabController,
  children: const [
    HomeScreen(),
    FriendsScreen(),
    ProfileScreen(),
    SettingsScreen(),
  ],
), // TabBarView

// 🔥 BOTTOM TAB BAR
bottomNavigationBar: Container(
  color: const Color(0xFF121222),
  child: TabBar(
    controller: _tabController,
    labelColor: Colors.blueAccent,
    unselectedLabelColor: Colors.grey,
    indicatorColor: Colors.blueAccent,
    tabs: const [
      Tab(icon: Icon(Icons.home), text: "Home"),
      Tab(icon: Icon(Icons.people), text: "Friends"),
      Tab(icon: Icon(Icons.person), text: "Profile"),
      Tab(icon: Icon(Icons.settings), text: "Settings"),
    ],
  ), // TabBar
```

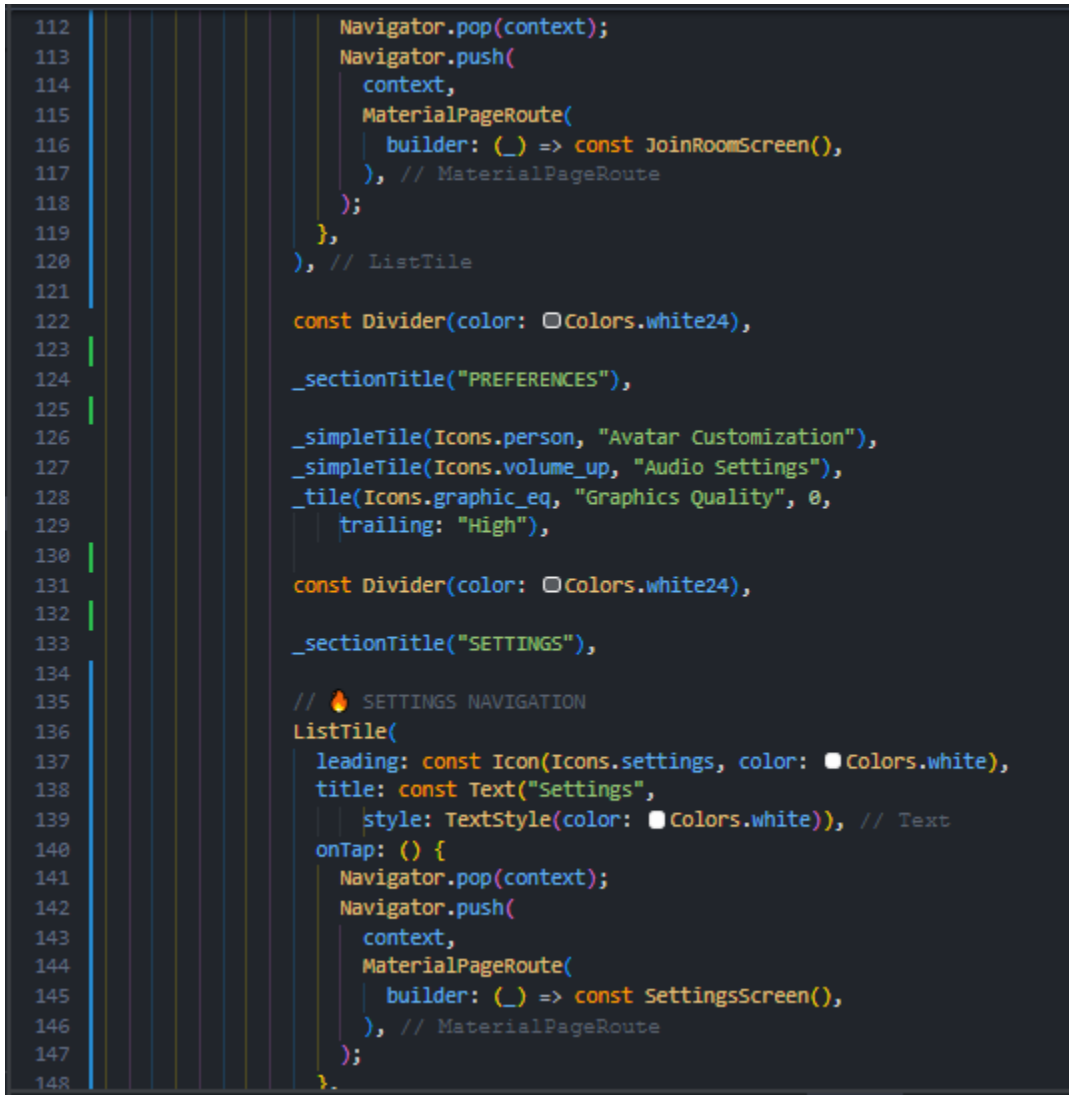
Drawer:

```dart
import 'package:flutter/material.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';

import '../screens/create_room_screen.dart';
import '../screens/join_room_screen.dart';
import '../screens/settings_screen.dart';

class AppDrawer extends StatelessWidget {
  final TabController tabController;

  const AppDrawer({super.key, required this.tabController});

  @override
  Widget build(BuildContext context) {
    final user = FirebaseAuth.instance.currentUser;

    return Drawer(
      backgroundColor: const Color(0xFF0F0F1C),
      child: Column(
        children: [
          // 🔥 HEADER
          StreamBuilder<DocumentSnapshot>(
            stream: FirebaseFirestore.instance
                .collection("users")
                .doc(user!.uid)
                .snapshots(),
            builder: (context, snapshot) {
              if (!snapshot.hasData) {
                return const SizedBox(
                  height: 200,
                  child: Center(child: CircularProgressIndicator()),
                ); // SizedBox
              }

              var data = snapshot.data!.data() as Map<String, dynamic>;
```



```
return Container(
  width: double.infinity,
  padding:
      const EdgeInsets.symmetric(vertical: 30, horizontal: 20),
  decoration: const BoxDecoration(
    gradient: LinearGradient(
      colors: [Color(0xFF1A1A2E), Color(0xFF16213E)],
    ), // LinearGradient
  ), // BoxDecoration
  child: Column(
    children: [
      const CircleAvatar(
        radius: 35,
        child: Icon(Icons.person, size: 40),
      ), // CircleAvatar
      const SizedBox(height: 10),
      Text(
        data['name'] ?? "No Name",
        style: const TextStyle(
          color: Colors.white,
          fontSize: 18,
          fontWeight: FontWeight.bold,
        ), // TextStyle
      ), // Text
      const SizedBox(height: 5),
      Text(
        data['email'] ?? "No Email",
        style: const TextStyle(
          color: Colors.white70,
          fontSize: 13,
        ), // TextStyle
      ), // Text
    ],
  ), // Column
); // Container
},
), // StreamBuilder
```

```
75
76          // 🔥 MENU
77          Expanded(
78            child: ListView(
79              padding: EdgeInsets.zero,
80              children: [
81                _sectionTitle("MAIN"),
82                _tile(Icons.home, "Home", 0),
83                _tile(Icons.people, "Friends", 1),
84                _tile(Icons.history, "History", 0),
85
86                const Divider(color: Colors.white24),
87
88                _sectionTitle("QUICK ACTIONS"),
89
90                // 🔥 CREATE ROOM NAVIGATION
91                ListTile(
92                  leading: const Icon(Icons.add, color: Colors.white),
93                  title: const Text("Create Room",
94                      style: TextStyle(color: Colors.white)), // Text
95                  onTap: () {
96                    Navigator.pop(context);
97                    Navigator.push(
98                      context,
99                      MaterialPageRoute(
100                       builder: (_) => const CreateRoomScreen(),
101                     ), // MaterialPageRoute
102                   );
103                 },
104               ), // ListTile
105
106               // 🔥 JOIN ROOM NAVIGATION
107               ListTile(
108                 leading: const Icon(Icons.login, color: Colors.white),
109                 title: const Text("Join Room",
110                     style: TextStyle(color: Colors.white)), // Text
111                 onTap: () {
```

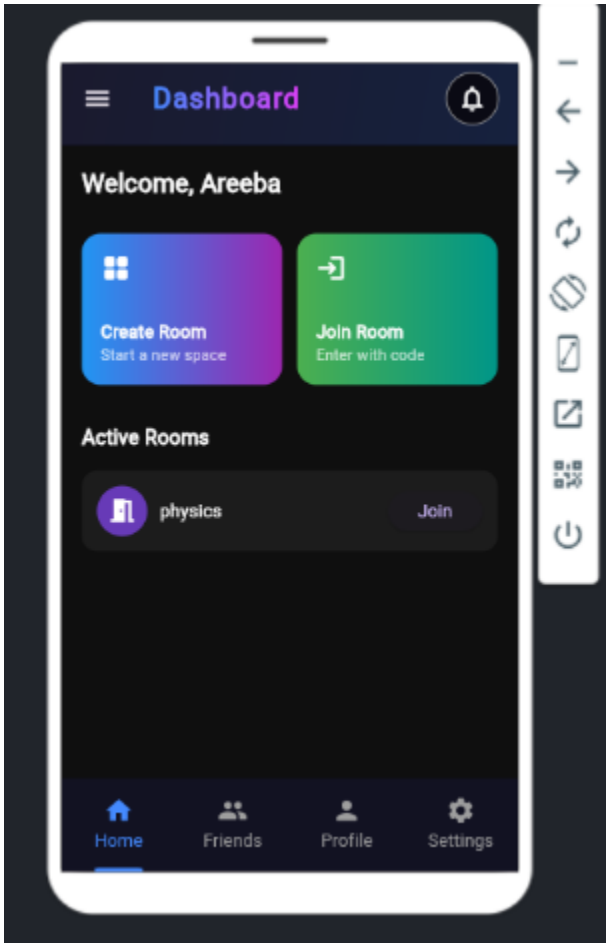


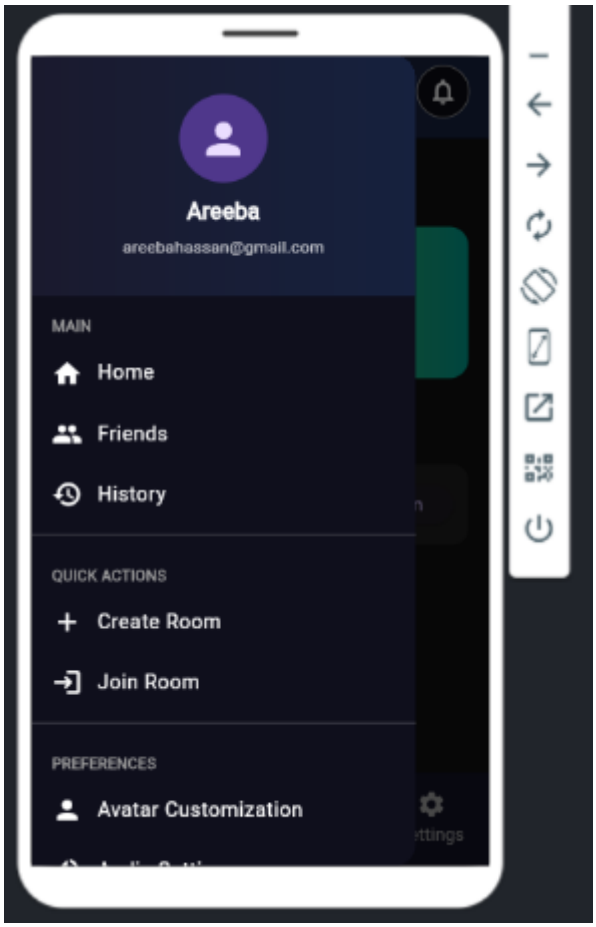
```
                Navigator.pop(context);
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (_) => const JoinRoomScreen(),
                  ), // MaterialPageRoute
                );
              },
            ), // ListTile

            const Divider(color: Colors.white24),

            _sectionTitle("PREFERENCES"),

            _simpleTile(Icons.person, "Avatar Customization"),
            _simpleTile(Icons.volume_up, "Audio Settings"),
            _tile(Icons.graphic_eq, "Graphics Quality", 0,
                trailing: "High"),

            const Divider(color: Colors.white24),

            _sectionTitle("SETTINGS"),

            // 🔥 SETTINGS NAVIGATION
            ListTile(
              leading: const Icon(Icons.settings, color: Colors.white),
              title: const Text("Settings",
                  style: TextStyle(color: Colors.white)), // Text
              onTap: () {
                Navigator.pop(context);
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (_) => const SettingsScreen(),
                  ), // MaterialPageRoute
                );
              },
```

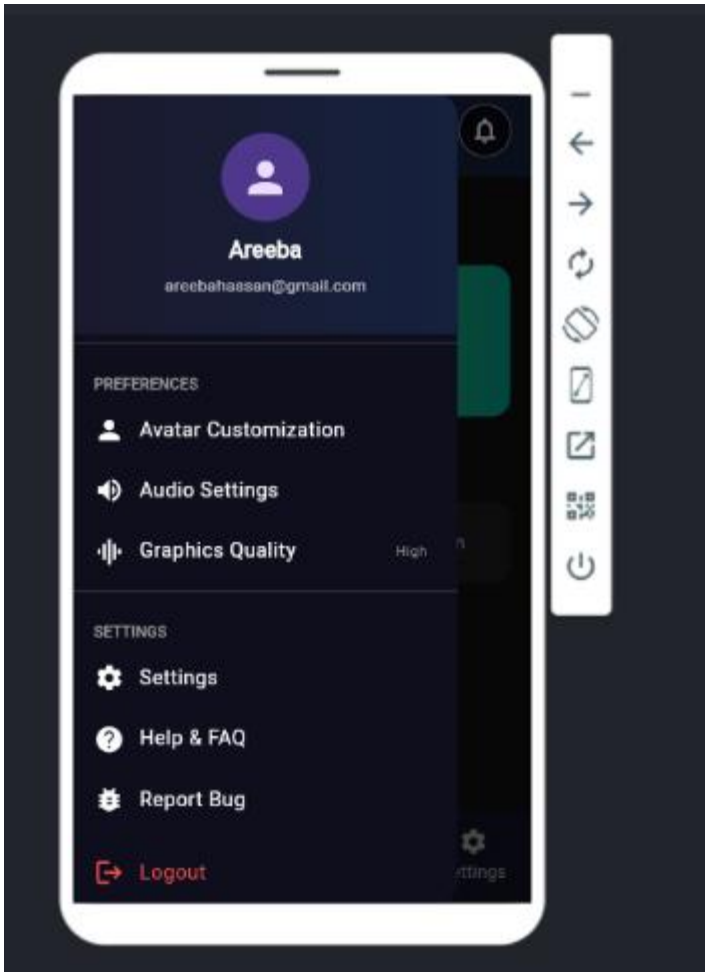
```
                ), // ListTile

                _simpleTile(Icons.help, "Help & FAQ"),
                _simpleTile(Icons.bug_report, "Report Bug"),

                const SizedBox(height: 10),

                ListTile(
                  leading: const Icon(Icons.logout, color: Colors.redAccent),
                  title: const Text("Logout",
                      style: TextStyle(color: Colors.redAccent)), // Text
                  onTap: () async {
                    await FirebaseAuth.instance.signOut();
                    Navigator.pop(context);
                  },
                ), // ListTile
              ],
            ), // ListView
          ), // Expanded
        ],
      ), // Column
    ); // Drawer
  }

  Widget _sectionTitle(String title) {
    return Padding(
      padding: const EdgeInsets.fromLTRB(16, 15, 16, 5),
      child: Text(
        title,
        style: const TextStyle(
          color: Colors.grey,
          fontSize: 12,
          fontWeight: FontWeight.bold,
        ), // TextStyle
      ), // Text
    ); // Padding
  }

  Widget _tile(IconData icon, String title, int index, {String? trailing}) {
    return ListTile(
      leading: Icon(icon, color: Colors.white),
      title: Text(title, style: const TextStyle(color: Colors.white)),
      trailing: trailing != null
          ? Text(trailing, style: const TextStyle(color: Colors.grey))
          : null,
      onTap: () {
        tabController.index = index;
      },
    ); // ListTile
  }

  Widget _simpleTile(IconData icon, String title) {
    return ListTile(
      leading: Icon(icon, color: Colors.white),
      title: Text(title, style: const TextStyle(color: Colors.white)),
      onTap: () {},
    ); // ListTile
  }
}
```

In dashboard

```
drawer: AppDrawer(tabController: _tabController),
```




Areeba
areebahassan@gmail.com
MAIN
Home
Friends
History
QUICK ACTIONS
Create Room
Join Room
PREFERENCES
Avatar Customization


Areeba
areebahaasan@gmail.com
PREFERENCES
Avatar Customization
Audio Settings
Graphics Quality
High
SETTINGS
Settings
Help & FAQ
Report Bug
Logout